



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

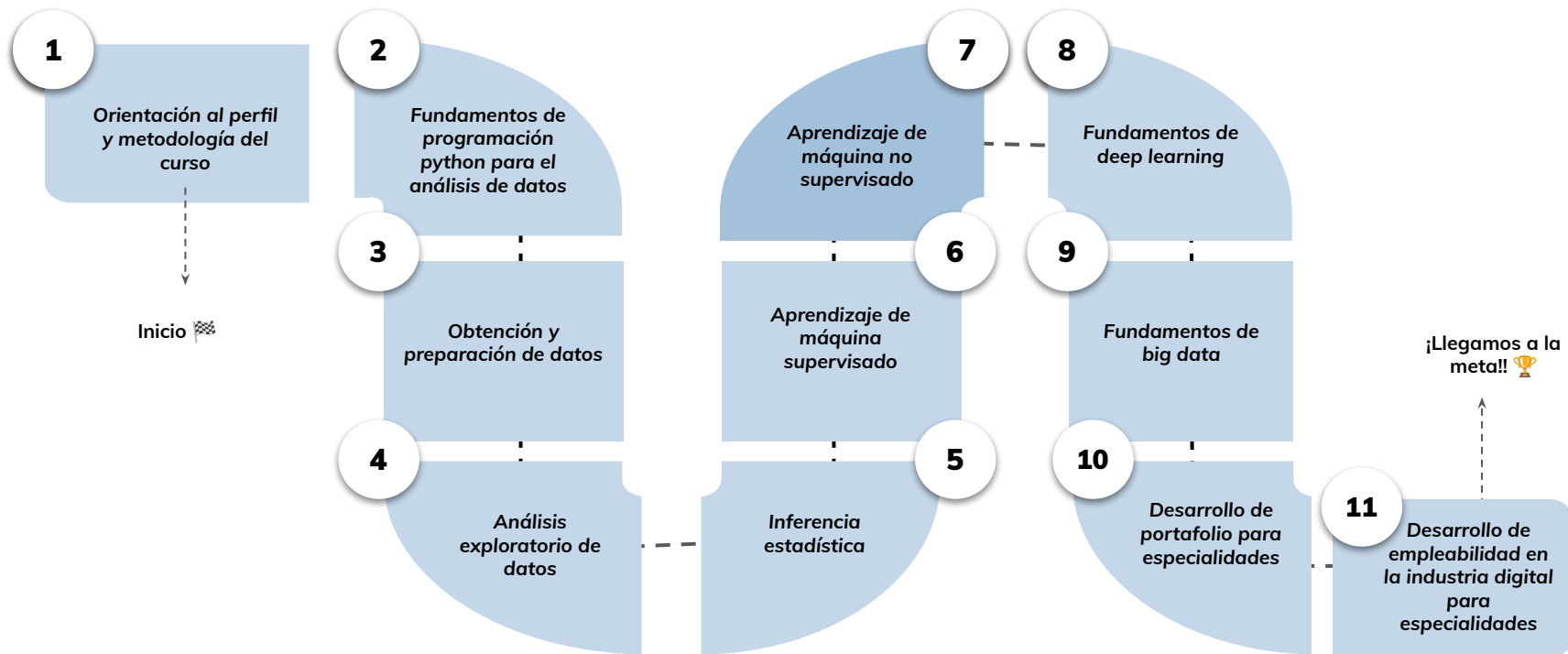


› Técnicas de reducción dimensional- Parte I

Aprendizaje Esperado 5: Implementar un modelo predictivo utilizando técnicas de reducción dimensional utilizando python para resolver un problema.

Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?

APRENDIZAJE DE MÁQUINA
NO SUPERVISADO

1

REDUCCIÓN
DIMENSIONAL

3

TÉCNICAS DE REDUCCIÓN
DIMENSIONAL

5

CLUSTERIZACIÓN Y SUS
PRINCIPALES ALGORITMOS

ALGORITMOS DE
CLUSTERIZACIÓN

Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Aplicación práctica de técnicas de reducción de dimensionalidad

Uso de métodos como PCA y t-SNE para transformar datasets complejos en representaciones más simples que mejoren visualización, rendimiento y análisis predictivo.

Implementación con Python

Comparación y evaluación

Preprocesamiento, escalado y reducción con sklearn

Visualización de resultados con PCA y t-SNE

Cuándo elegir PCA o t-SNE según el problema

Análisis de métricas, ventajas y desafíos



Objetivos de aprendizaje

¿Qué aprenderás?



- Implementar técnicas de reducción dimensional en Python con scikit-learn
- Pre Procesar correctamente un dataset antes de aplicar la reducción
- Visualizar los datos transformados para facilitar su análisis
- Comparar los resultados obtenidos con PCA y t-SNE
- Identificar cuándo usar una técnica u otra según el caso de uso

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Qué es la reducción de dimensionalidad y para qué se utiliza
- Tipos de técnicas: lineales (PCA) y no lineales (t-SNE)
- Fundamentos matemáticos de PCA y cómo reduce la varianza
- Cómo t-SNE preserva relaciones locales en visualizaciones
- Ventajas y limitaciones de ambas técnicas

Técnicas de reducción dimensional



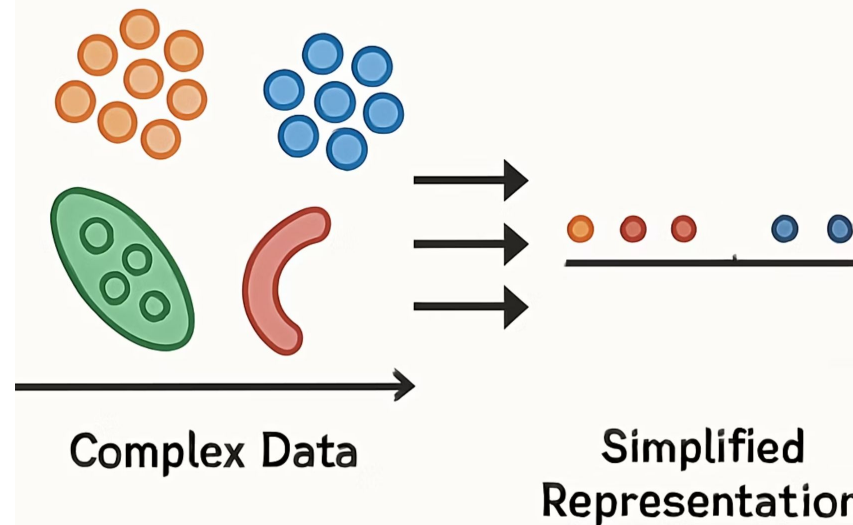
Técnicas de Reducción de Dimensionalidad en Aprendizaje Automático

En el mundo del aprendizaje automático, los conjuntos de datos suelen tener una gran cantidad de variables o características, lo cual puede dificultar el análisis, aumentar los tiempos de procesamiento y llevar a problemas como el sobreajuste. Para abordar estos inconvenientes, se emplean técnicas de reducción de dimensionalidad que permiten simplificar los datos sin perder información relevante.

Introducción a la Reducción de Dimensionalidad

La reducción de dimensionalidad es especialmente útil en tareas de visualización, compresión de datos y mejora de la eficiencia de los algoritmos. En este manual abordaremos dos de las técnicas más utilizadas: PCA (Análisis de Componentes Principales) y t-SNE (t-distributed Stochastic Neighbor Embedding).

MACHINE LEARNING DIMENSIONALITY REDUCTION



Objetivos del Manual

Aprendizaje de Técnicas

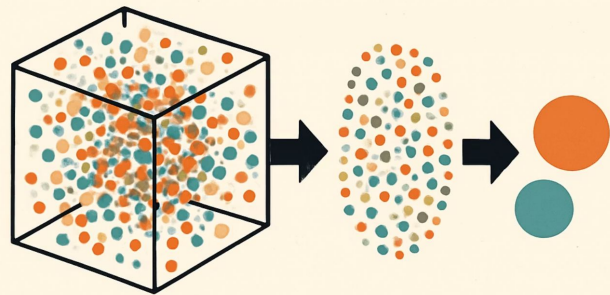
Aprenderás cuándo conviene aplicar cada método, sus ventajas, limitaciones y cómo implementarlos en Python usando bibliotecas como scikit-learn.

Ejemplos Prácticos

Con ejemplos prácticos y visuales, vas a poder identificar la técnica adecuada según tus datos y objetivos analíticos.

Técnicas de reducción dimensional

Las técnicas de reducción dimensional permiten transformar datos complejos en representaciones más simples y manejables, facilitando su análisis y procesamiento.



DIMENSIONALITY REDUCTION

¿Qué es la reducción de dimensionalidad?

La reducción de dimensionalidad es una técnica utilizada en ciencia de datos para disminuir el número de variables en un conjunto de datos, manteniendo al mismo tiempo la mayor cantidad de información relevante posible. Se emplea principalmente cuando los datos tienen muchas características, lo que puede dificultar el análisis e impactar negativamente en la eficiencia de los modelos.

Beneficios de la Reducción de Dimensionalidad

Visualización Mejorada

Facilita la visualización de los datos, ya que reduce las variables a dos o tres dimensiones comprensibles por humanos.

Mayor Rendimiento

Mejora el rendimiento de los algoritmos de machine learning al disminuir la complejidad computacional.

Reducción de Sobreajuste

Puede ayudar a combatir el sobreajuste eliminando ruido y redundancia.

Tipos de Técnicas

1

Técnicas Lineales

Como PCA, asumen relaciones lineales entre variables y son generalmente más rápidas y sencillas de implementar.

2

Técnicas No Lineales

Como t-SNE, pueden capturar relaciones complejas no lineales entre variables, pero suelen ser más costosas computacionalmente.

La elección adecuada dependerá de la estructura de los datos y del objetivo del análisis.

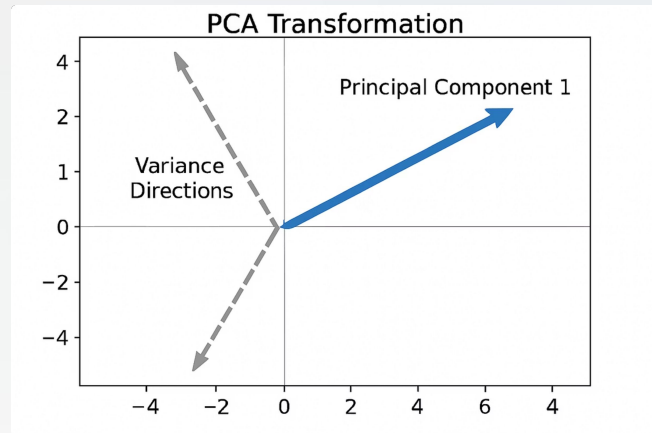


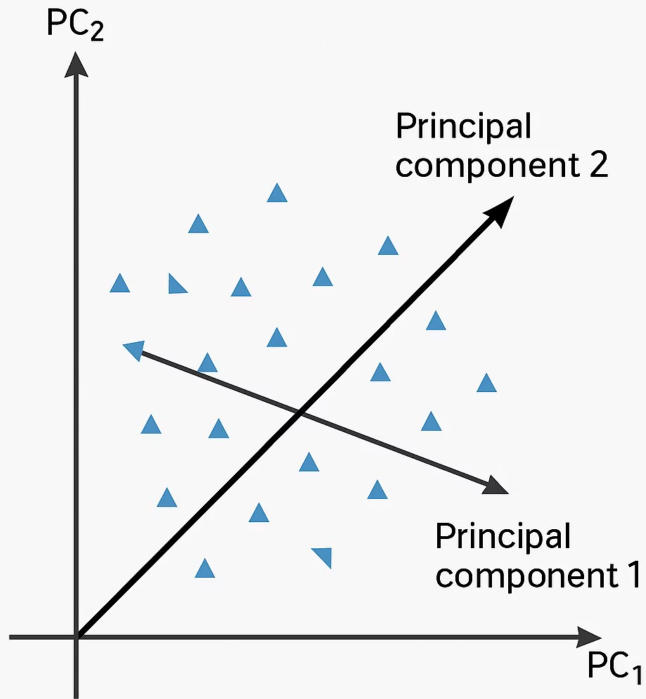
Consideraciones Importantes

Es fundamental aplicar reducción de dimensionalidad de forma cuidadosa, evaluando el impacto de cada técnica en la estructura de los datos y su interpretabilidad posterior.

Análisis de Componentes Principales (PCA)

PCA (Principal Component Analysis) es una técnica estadística que transforma un conjunto de variables correlacionadas en un nuevo conjunto de variables no correlacionadas llamadas componentes principales.





Fundamentos de PCA

Estas nuevas variables representan direcciones de máxima varianza en los datos, permitiendo así conservar la mayor parte de la información en menos dimensiones.

Base Matemática de PCA

Cálculo de la Media

Se calcula la media de cada variable en el conjunto de datos.

Descomposición en Valores Propios

Se calculan los vectores y valores propios de la matriz de covarianza.

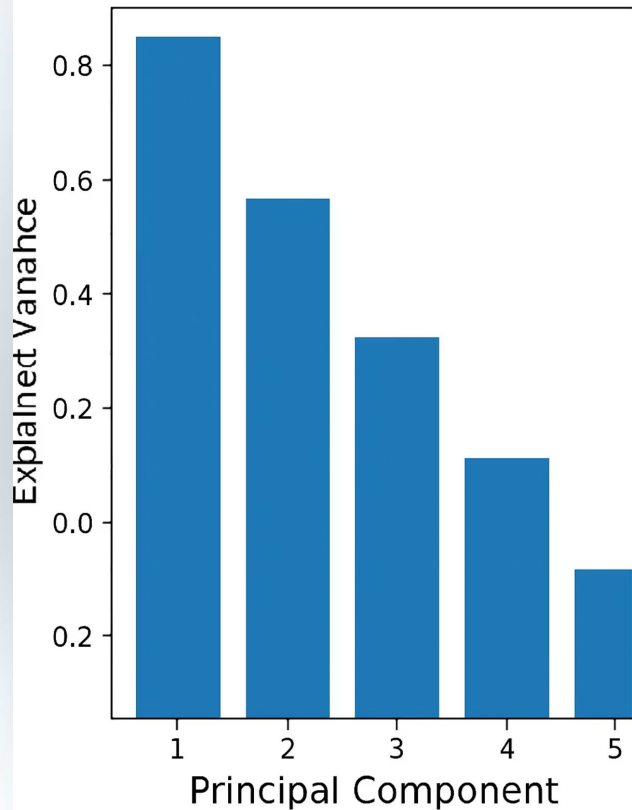
Matriz de Covarianza

Se construye la matriz que representa las relaciones entre variables.

Selección de Componentes

Se eligen los componentes que explican la mayor varianza.

Decreasing Explained
Variance by Principal Component



Componentes Principales

Los primeros componentes principales explican la mayor cantidad de varianza, por lo que suelen bastar unos pocos para representar adecuadamente los datos.

Ventajas de PCA



Eficiencia Computacional

PCA es relativamente rápido incluso con grandes conjuntos de datos.



Simplicidad de Implementación

Es fácil de implementar con bibliotecas estándar de ciencia de datos.



Eliminación de Redundancia

Elimina eficazmente la información redundante entre variables correlacionadas.

Limitaciones de PCA



Relaciones No Lineales

Incapacidad para captar relaciones no lineales entre variables.



Interpretabilidad

Dificultad para interpretar los componentes en términos de las variables originales.



Pérdida de Información

Puede descartar información importante si está contenida en componentes con baja varianza.

Ejemplo en Python: PCA

Este código muestra cómo implementar PCA en Python utilizando la biblioteca scikit-learn, incluyendo la visualización de los resultados.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.title("PCA: Reducción a 2 dimensiones")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Ventajas vs Desventajas de PCA

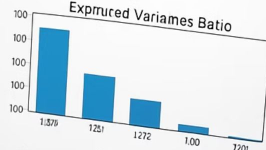
Ventajas	Desventajas
Rápido y fácil de implementar	Solo captura relaciones lineales
Reduce redundancia	Difícil interpretación de los componentes
Mejora la eficiencia	Puede perder información importante

PCA for Exploratory Data Analysis

Explained Variables

Точность составной модели	Точность базисной
0.00	0.00
0.500	0.00
1.00	0.00

Explained Variables Ratio



PCA Active Pax



PCS

Re-ent	4.00	4.60
3h	10.00	7.00
3d	8.00	18.00
3h	11.00	5.50
3h	2.00	16.00
3h	2.60	6.00
3h	19.00	10.00

Recomendaciones de Uso para PCA

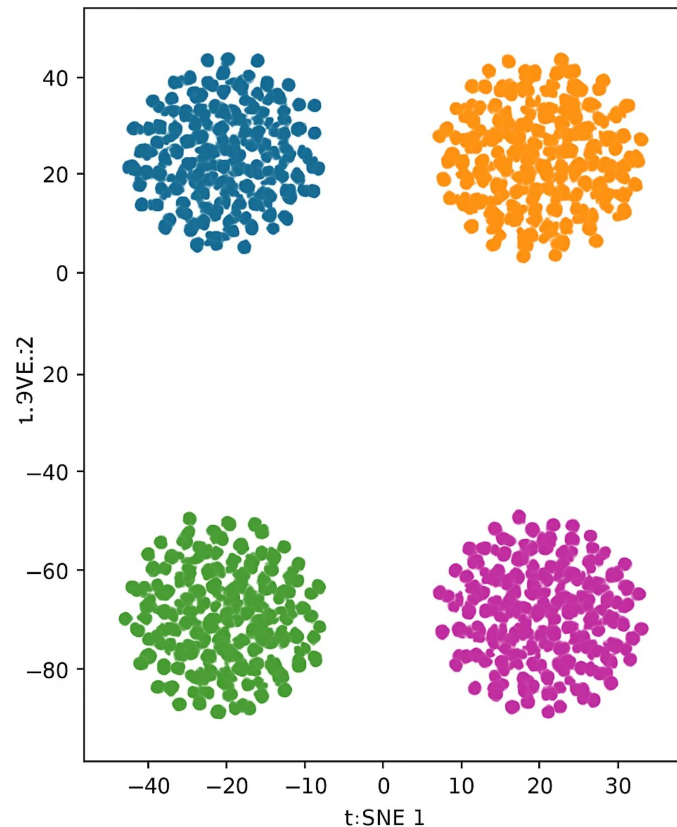


PCA se recomienda en fases tempranas de exploración, compresión de datos o como paso previo a otros modelos.

t-Distributed Stochastic Neighbor Embedding (t-SNE)

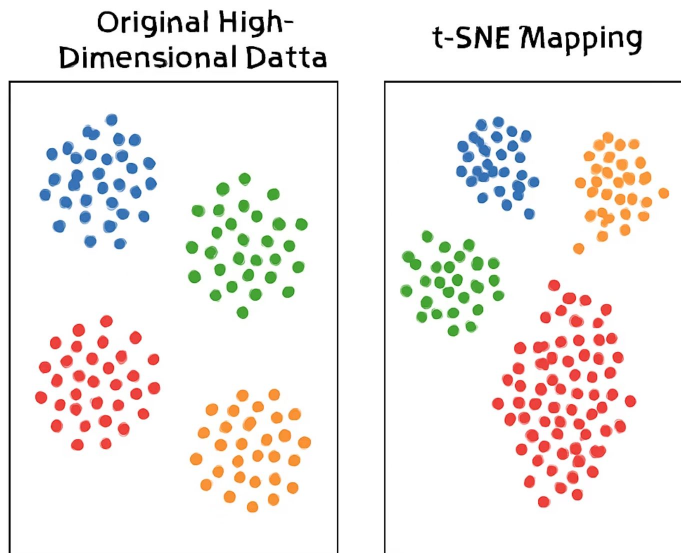
t-SNE es una técnica no lineal desarrollada para la visualización de datos de alta dimensión.

t-SNE visualization showing clusters of high-dimensional data in 2D space with clear separation



Fundamentos de t-SNE

A diferencia de PCA, que busca conservar la varianza, t-SNE se enfoca en preservar las relaciones de proximidad locales entre puntos. Utiliza una métrica probabilística para mapear datos en espacios de menor dimensión, generalmente 2D o 3D.



Funcionamiento de t-SNE

Cálculo de Similitudes

Calcula la similitud entre puntos en el espacio original usando distribuciones de probabilidad gaussianas.

Optimización

Ajusta iterativamente las posiciones para que las similitudes en el espacio reducido coincidan con las originales.

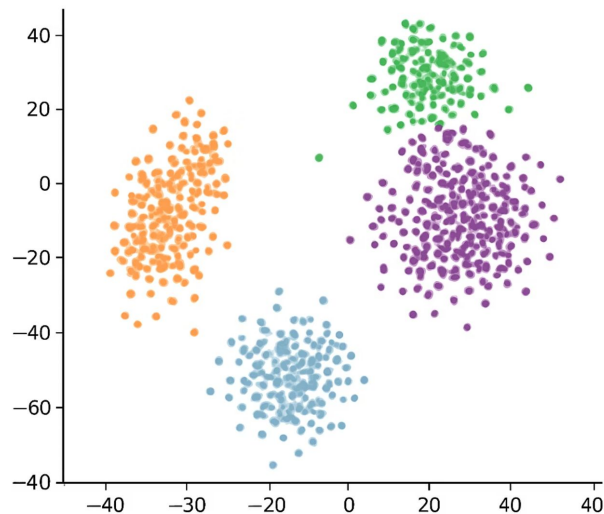
Creación del Espacio Reducido

Crea un espacio de baja dimensión donde los puntos se distribuyen inicialmente al azar.

Distribución t-Student

Usa la distribución t-Student para evitar el "problema de aglomeración" en el espacio reducido.

t-SNE Revealing Hidden
Clusters in Genomic Data



Fortalezas de t-SNE

La principal fortaleza de t-SNE radica en su capacidad para revelar agrupaciones ocultas y estructuras complejas que otros métodos no detectan. Es ampliamente utilizado en bioinformática, análisis de texto y visión por computadora.

Ejemplo en Python: t-SNE

Este código muestra cómo implementar t-SNE en Python utilizando la biblioteca scikit-learn, incluyendo la visualización de los resultados.

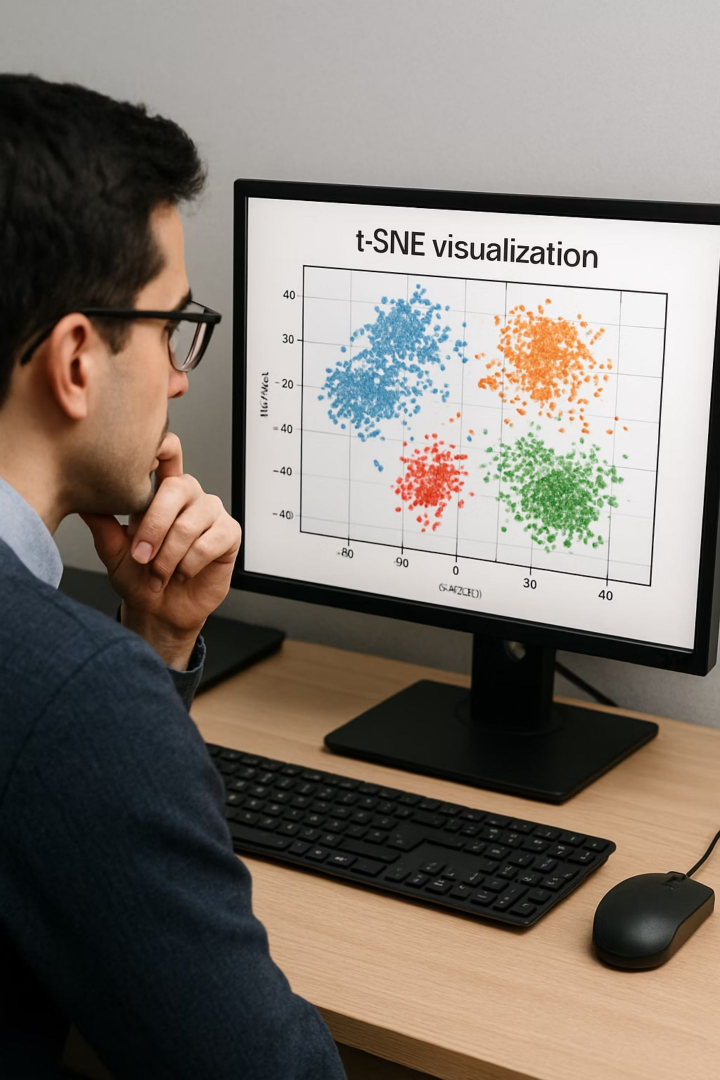
```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X)

plt.scatter(X_tsne[:, 0], X_tsne[:, 1])
plt.title("t-SNE: Proyección bidimensional")
plt.xlabel("Dim 1")
plt.ylabel("Dim 2")
plt.show()
```

Ventajas vs Desventajas de t-SNE

Ventajas	Desventajas
Excelente para visualizar agrupamientos	No reutilizable en nuevos datos
Captura relaciones no lineales	Requiere ajustes finos de hiperparámetros
Funciona bien en datos ruidosos	Tiempo de ejecución elevado



Recomendaciones de Uso para t-SNE



Debido a su enfoque visual, t-SNE no es adecuado para usar como entrada a modelos predictivos, pero sí para entender mejor la distribución y relaciones en los datos.

Hiperparámetros Importantes en t-SNE

Perplejidad

Controla el equilibrio entre la preservación de estructuras locales y globales. Valores típicos entre 5 y 50.

Número de Iteraciones

Determina cuánto tiempo se optimiza el algoritmo. Más iteraciones pueden dar mejores resultados pero aumentan el tiempo de ejecución.

Tasa de Aprendizaje

Controla la velocidad de convergencia. Un valor demasiado alto puede causar inestabilidad, mientras que uno demasiado bajo puede resultar en mínimos locales.

Aplicaciones Comunes de t-SNE



Bioinformática

Visualización de datos genómicos y proteómicos para identificar patrones en expresión génica.



Procesamiento de Lenguaje Natural

Visualización de embeddings de palabras y documentos para análisis semántico.



Visión por Computadora

Visualización de características extraídas de imágenes para clasificación y reconocimiento de patrones.

Comparación Visual: PCA vs t-SNE

La imagen muestra cómo t-SNE suele revelar agrupaciones más claras que PCA, especialmente en datos con estructuras no lineales.

Cuándo Usar PCA vs t-SNE



Usa PCA cuando...

- Necesitas reducir dimensiones rápidamente
- Quieres preservar la varianza global
- Necesitas proyectar nuevos datos
- Buscas compresión de datos



Usa t-SNE cuando...

- Necesitas visualizar agrupaciones
- Buscas preservar estructuras locales
- Sospechas relaciones no lineales
- El tiempo de ejecución no es crítico

Live Coding

¿En qué consistirá la Demo?

Implementaremos dos técnicas de reducción de dimensionalidad en Python: PCA y t-SNE. El foco estará en observar cómo transformar un conjunto de datos de alta dimensión a dos dimensiones para facilitar su análisis y visualización.

1. Importación de librerías (pandas, numpy, sklearn, matplotlib)
2. Carga del dataset (por ejemplo: iris.csv o uno multivariable real)
3. Preprocesamiento: limpieza, normalización con StandardScaler
4. Aplicación de PCA para reducir a 2D y visualizar componentes principales
5. Aplicación de t-SNE y comparación visual con los resultados de PCA
6. Discusión guiada:
 - a. ¿Qué diferencias se observan?
 - b. ¿Qué técnica preserva mejor la estructura local?
 - c. ¿Cuál facilita mejor la interpretación visual?



Momento:

Time-out!



5 -10 min.





Ejercicio

Reduciendo para entender mejor

Reduciendo para entender mejor

Contexto: 🙌

Trabajas en un equipo que desarrolla modelos de clasificación. Antes de entrenar el modelo final, quieren saber si reducir la cantidad de variables mejora el rendimiento y la visualización.

Consigna: 📝

Implementa un flujo de trabajo completo que combine reducción de dimensionalidad con un modelo de clasificación supervisada.

Paso a paso: ⚙️

1. Carga un dataset multiclase (sugerido: wine.csv o digits de sklearn.datasets)
2. Dividelo en train y test
3. Estandariza los datos con StandardScaler
4. Entrena un modelo de clasificación base (por ejemplo, LogisticRegression o KNeighborsClassifier)
5. Calcula la accuracy en test sin reducción
6. Aplica PCA para reducir a 5 componentes
7. Reentrena el modelo con los datos reducidos
8. Compara la nueva accuracy con la original
9. Visualiza los datos reducidos en 2D (si es posible)
10. Concluye:
 - ¿Cambió el rendimiento?
 - ¿Reducir dimensiones ayuda o perjudica en este caso?
 - ¿En qué situaciones te parece útil este enfoque?

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Aplicamos en Python técnicas de reducción de dimensionalidad como PCA y t-SNE
- ✓ Preprocesamos datasets multivariables para facilitar su transformación
- ✓ Visualizamos datos en 2D para detectar patrones y estructuras ocultas
- ✓ Comparamos los resultados obtenidos con distintas técnicas



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

